

第五章 深度学习与神经网络

本课程目标



了解什么是深度学习与神经网络



了解人工神经网络的结构和相关概念



掌握前馈神经网络的结构和相关概念



了解深度学习框架

课程目录

Course catalogue

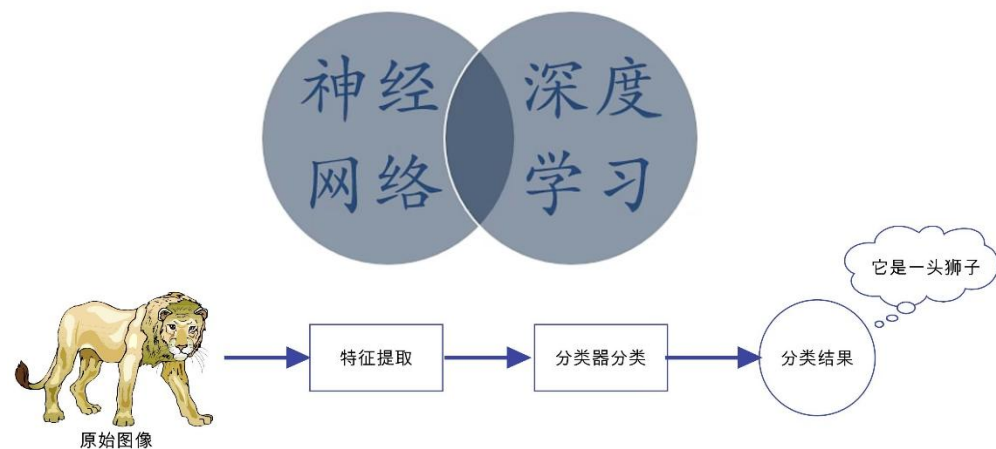
1/ 深度学习与神经网络

2/ 人工神经网络

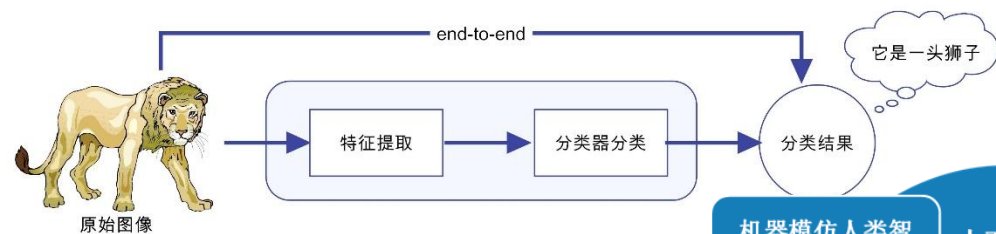
3/ 前馈神经网络

4/ 深度学习框架

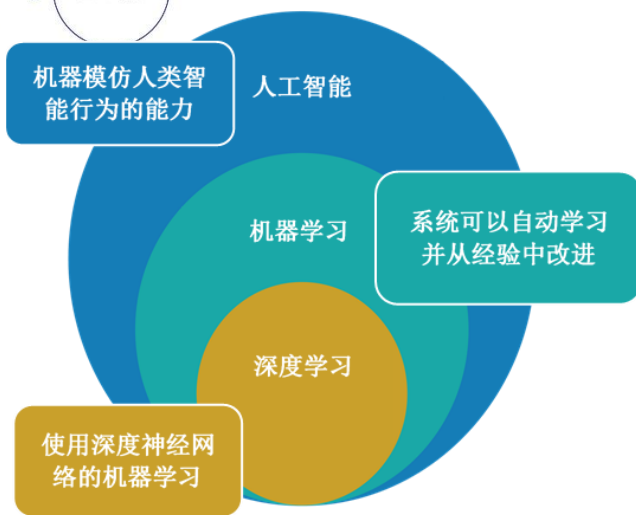
深度学习与神经网络



(a) 传统分类模式



(b) 深度神经网络分类模式



深度学习与神经网络

深度学习是机器学习的一种特殊形式，深度学习、机器学习和人工智能的关系如图所示，机器学习是人工智能的一个子领域，而深度学习又是机器学习的一个子领域。

课程目录

Course catalogue

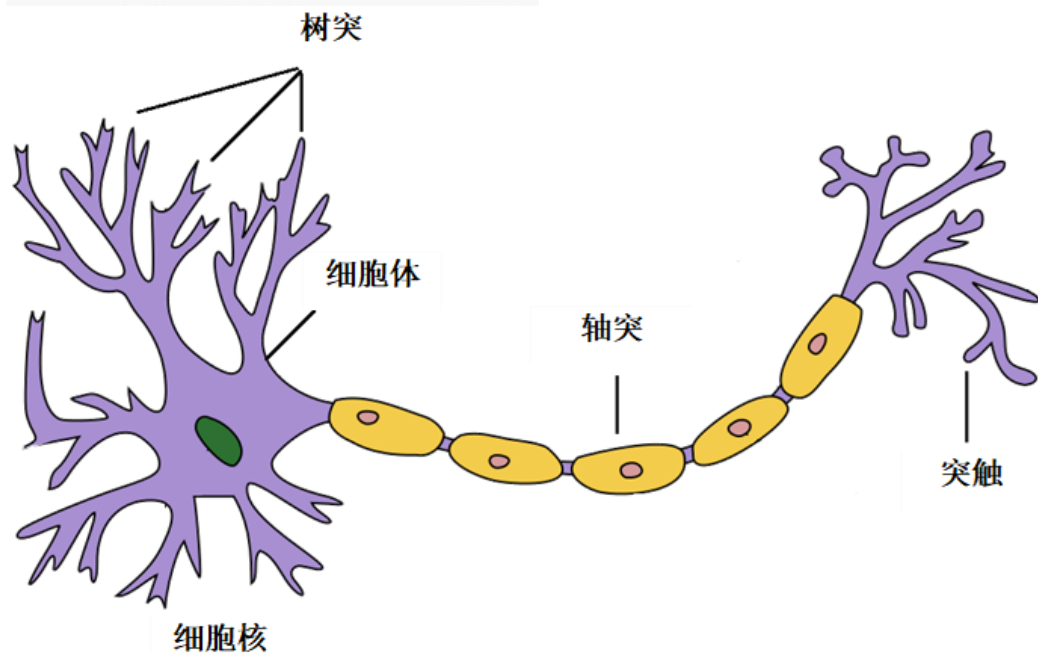
1/ 深度学习与神经网络

2/ 人工神经网络

3/ 前馈神经网络

4/ 深度学习框架

人工神经网络



生物神经元

典型的生物神经元大致可以分为四个部分：

- 树突 (Dendrite)，接收刺激并将其传入细胞体；
- 细胞体 (Soma)，处理从树突中接收到的信息；
- 轴突 (Axon)，把自身的兴奋从细胞体传送到其它神经元或组织；
- 突触 (Synapse)，是轴突和其他神经元树突之间的连接。

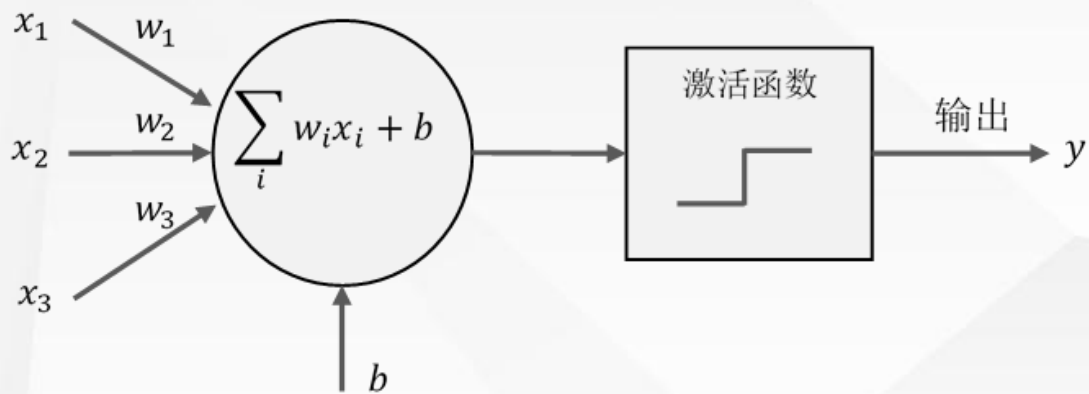
人工神经网络

感知器

感知器是一种模仿生物神经元构建的数学模型，它具有四个重要组成部分：输入、权重和偏置量、求和函数以及激活函数。

感知器是一个简单的数学模型，可以表示成如下形式：

$$y = \begin{cases} 1, & \sum_i w_i x_i + b \geq \text{阈值} \\ 0, & \text{其它} \end{cases}$$



激活函数

引入非线性函数作为激活函数，这样深层神经网络表达能力就更加强大，理论上只要神经元足够多，模型可以拟合任意函数。

激活函数需要具备以下几个性质：

➤ **连续且可导的非线性函数**

- 这样可以直接利用数值优化的方法来学习网络参数。

➤ **激活函数本身及其导函数的形式要尽可能简单**

这样有利于提高计算效率。

➤ **激活函数的导函数的值域要在一个合适的区间内**

- 不能太大或太小，避免影响训练的效果和稳定性。

常用激活函数

- S型函数

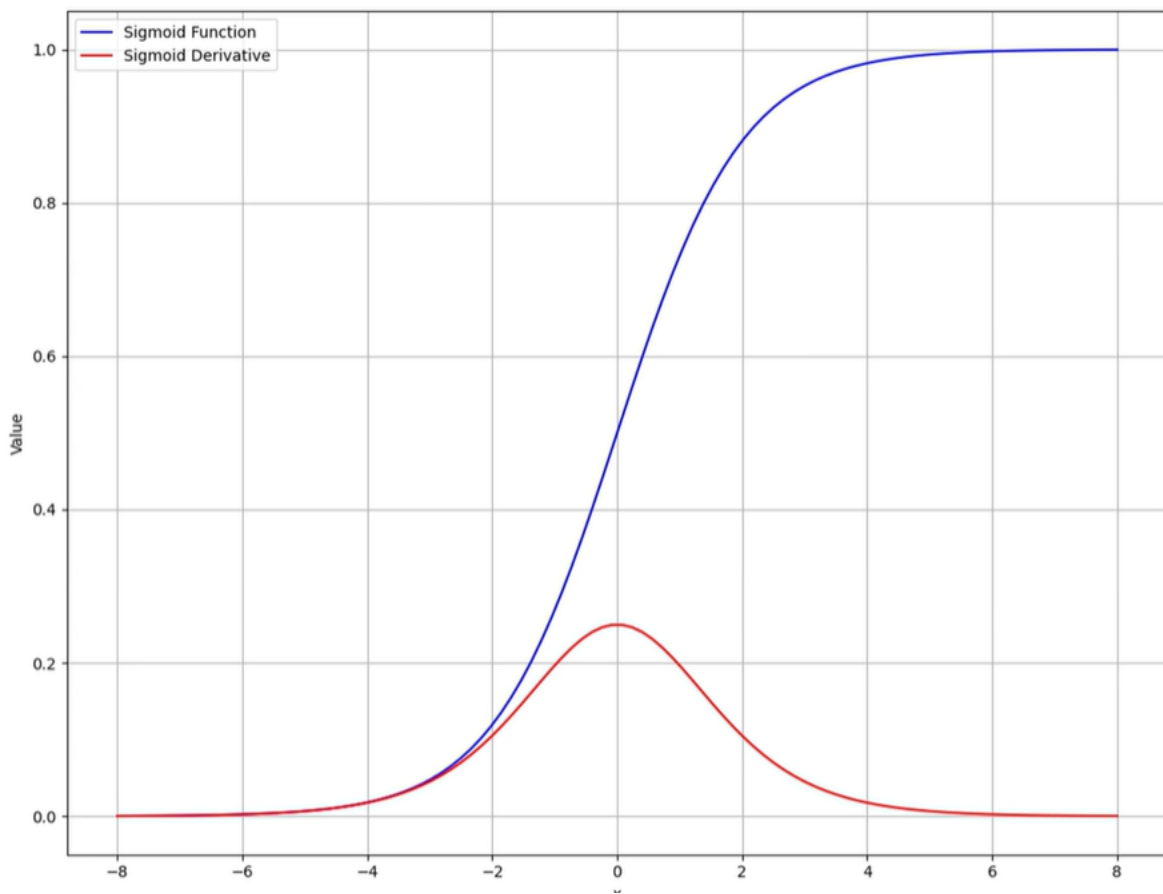
Sigmoid Function

- 斜坡函数

Ramp Function

- 复合函数

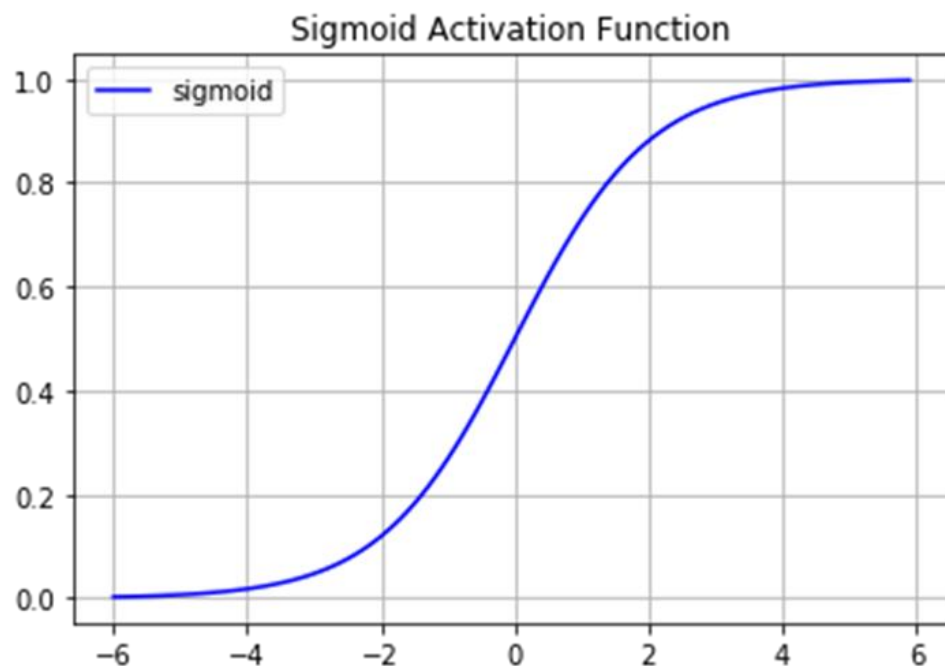
Compound Function



Logistic函数

Logistic函数是一个单调、有界、可导的实函数，其表达式如下：

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Logistic函数

Logistic函数是一个单调、有界、可导的实函数，其表达式如下：

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



有什么缺点？

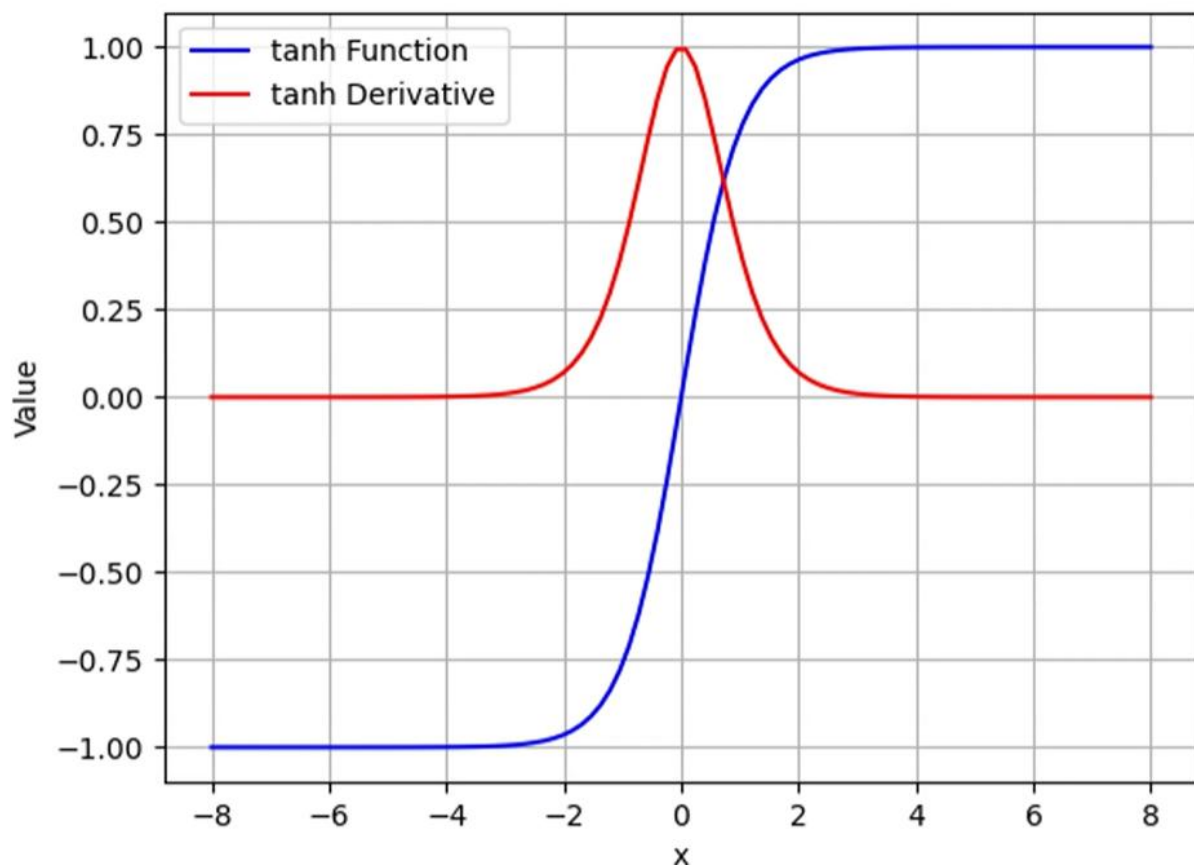
1. 梯度消失问题 (Vanishing Gradients)
2. 输出非零中心 (Non-Zero-Centered)
3. 函数饱和区导致的训练停滞
4. 计算复杂度较高

Tanh函数

Tanh函数称为双正切函数，其表达式为：

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

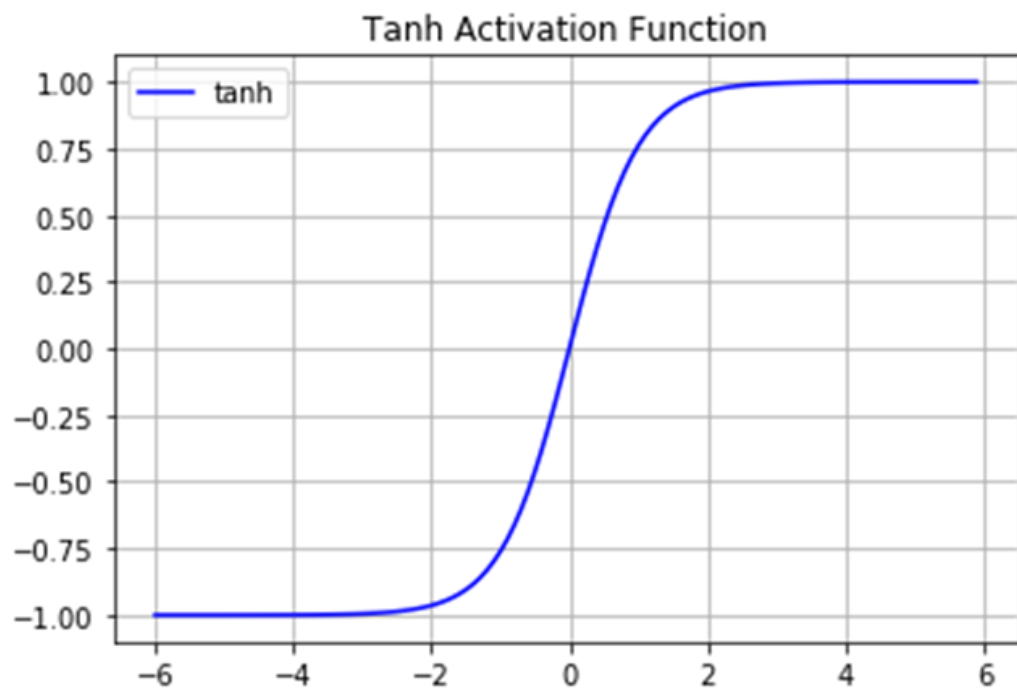
$$\tanh(x) = 2\delta(2x) - 1$$



Tanh函数

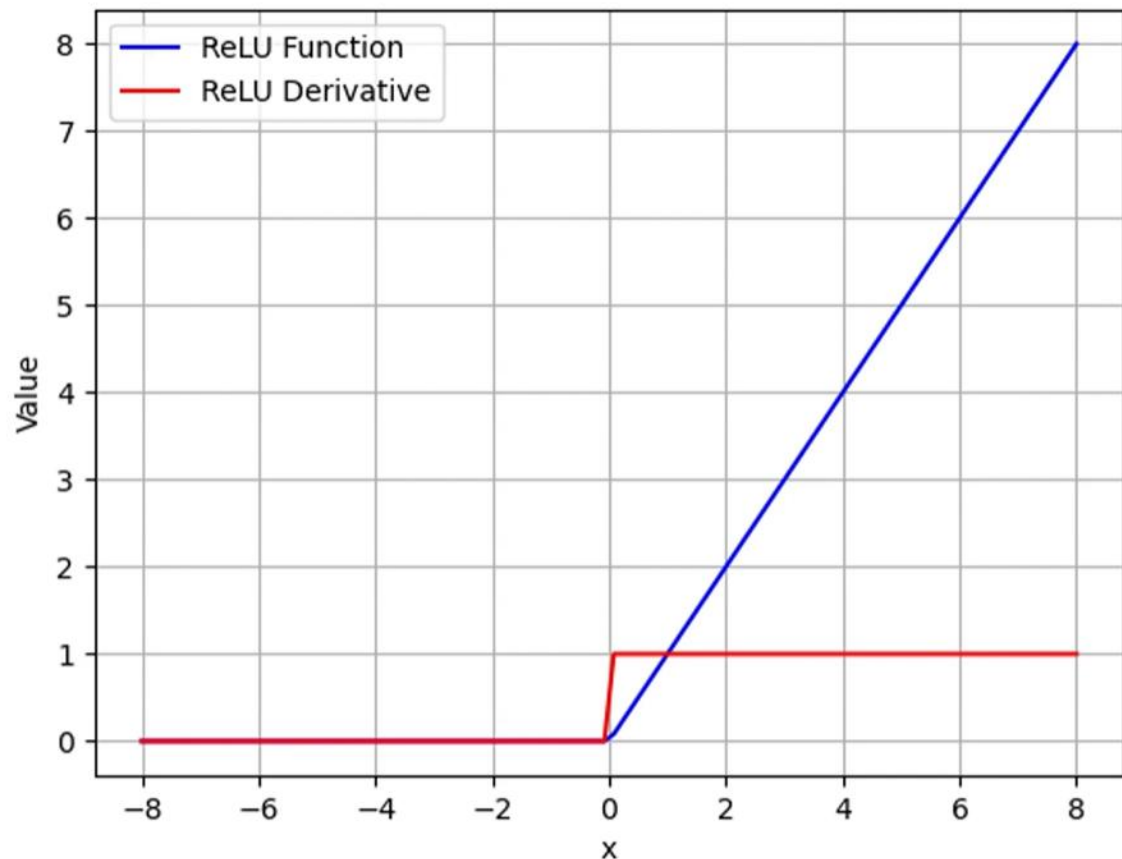
Tanh函数称为双正切函数，其表达式为：

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



有什么缺点？

1. 梯度消失问题 (Vanishing Gradients)
2. 函数饱和区导致的训练停滞
3. 计算复杂度较高



ReLU函数

ReLU (Rectified Linear Unit, 修正线性单元) 是近年来被大量使用的激活函数, 其表达式为:

$$ReLU(x) = \max(0, x)$$

ReLU函数

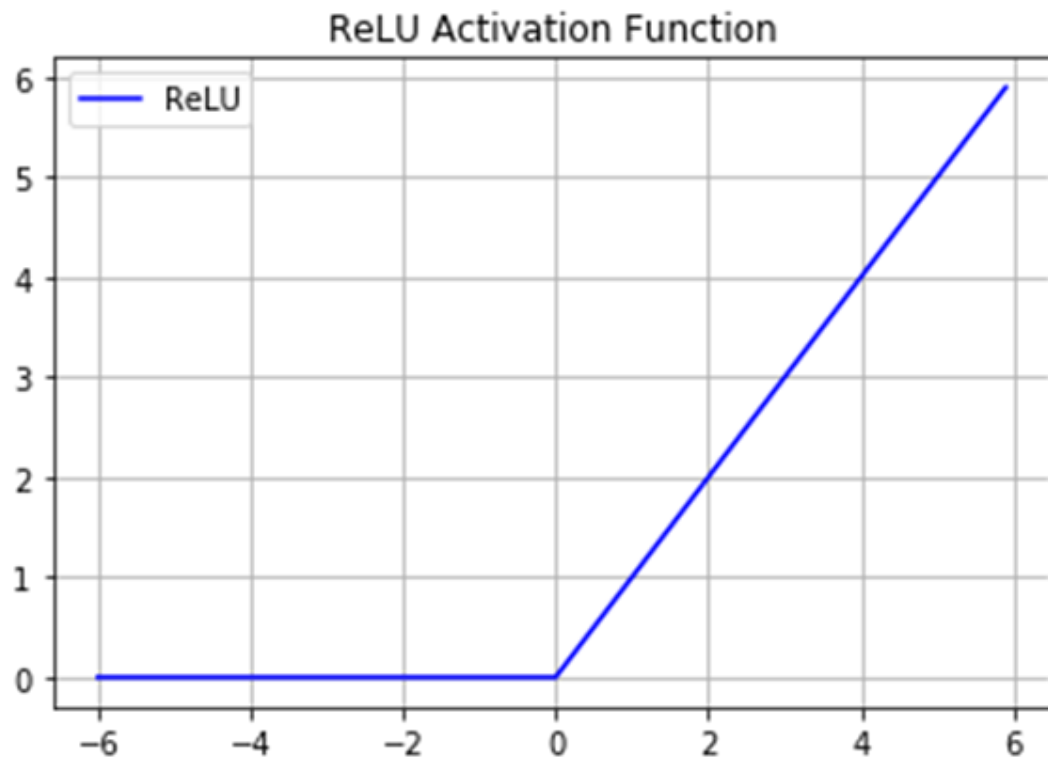
ReLU (Rectified Linear Unit, 修正线性单元) 是近年来被大量使用的激活函数, 其表达式为:

$$\text{ReLU}(x) = \max(0, x)$$

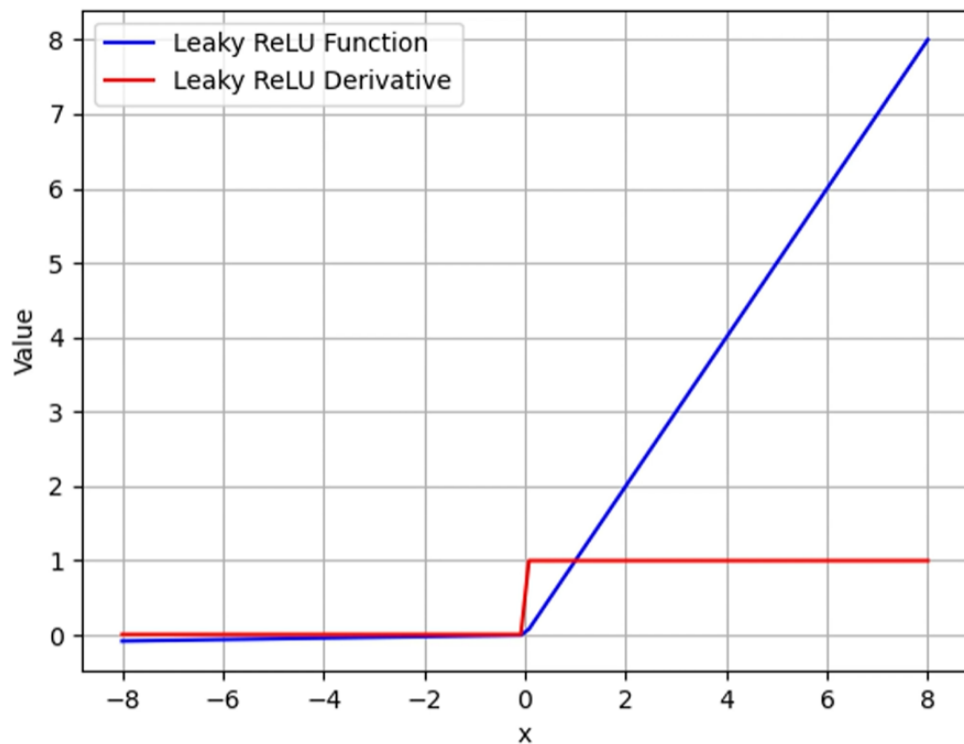


有什么缺点?

1. 神经元死亡 (Dead ReLU Problem)
2. 输出非零中心 (Non-Zero-Centered)
3. 对噪声敏感
4. 负区间完全失效



Leaky ReLU函数

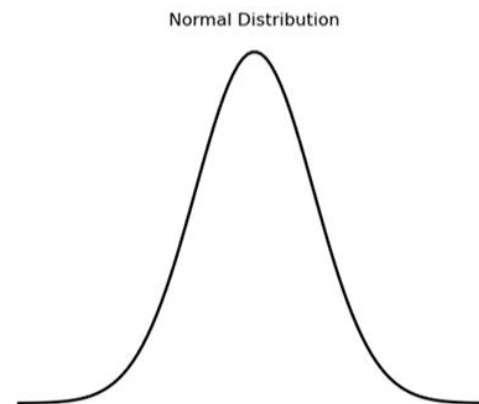


$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

0.01

随机 Leaky ReLU

Parametric ReLU



GELU 激活函数

什么是 GELU?

Gaussian Error Linear Unit (高斯误差线性单元)

$$\text{GELU}(x) = x \times \Phi(x)$$

其中 $\Phi(x)$ 是标准正态分布的累积分布函数。
直觉：根据输入值的大小，以某个概率“保留”或“干掉”它。

为什么重要?

- ✓ Transformer/BERT/GPT 的默认激活函数
- ✓ 平滑过渡，没有 ReLU 的“硬拐角”
- ✓ 允许少量负值通过，信息保留更好
- ✓ 实验表明在NLP任务中效果优于ReLU

GELU vs ReLU 对比

维度	ReLU	GELU
负值处理	完全截断（输出0）	允许少量负值通过（概率性）
光滑性	在 $x=0$ 处不可导	全程光滑可导
应用场景	CNN、传统网络	Transformer、BERT、GPT、ViT
计算成本	极低	稍高（涉及正态分布计算）

Swish / SiLU 激活函数

Swish 函数

由 Google Brain 团队于 2017 年通过自动搜索发现

Swish(x) = x × σ(βx)

当 β=1 时，称为 SiLU (Sigmoid Linear Unit)

SiLU(x) = x × σ(x) = x / (1 + e^{-x})

特点与应用

- ✓ 光滑、非单调，允许少量负值通过
- ✓ 效果介于 ReLU 和 GELU 之间
- ✓ EfficientNet、YOLOv5 等模型采用
- ✓ PyTorch: torch.nn.SiLU()

激活函数全景对比总结

激活函数	公式	优点	缺点	主要用途
Sigmoid	σ(x)=1/(1+e ^{-x})	平滑、有界	梯度消失、非零中心	二分类输出层
Tanh	tanh(x)	零中心	梯度消失	早期RNN、LSTM门控
ReLU	max(0,x)	计算快、缓解梯度消失	神经元死亡	CNN、默认选择
GELU	x×Φ(x)	光滑、保留负值信息	计算稍慢	Transformer、BERT
SiLU	x×σ(x)	光滑、自门控	计算稍慢	EfficientNet、YOLOv5

课程目录

Course catalogue

1/ 深度学习与神经网络

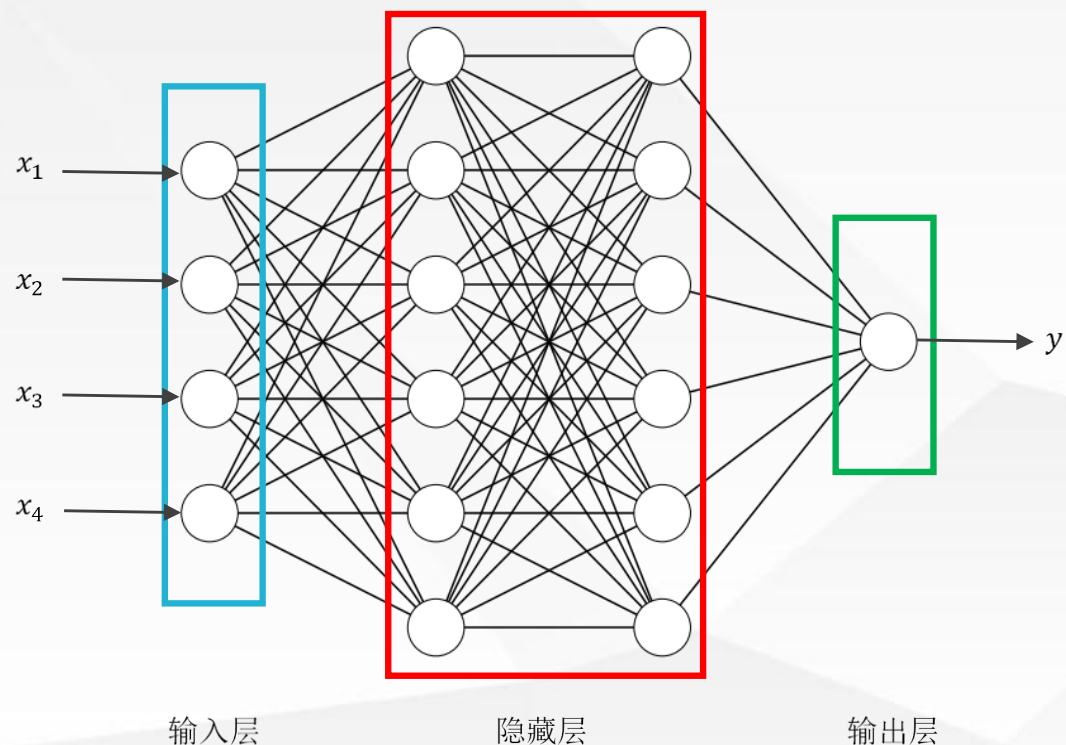
2/ 人工神经网络

3/ 前馈神经网络

4/ 深度学习框架

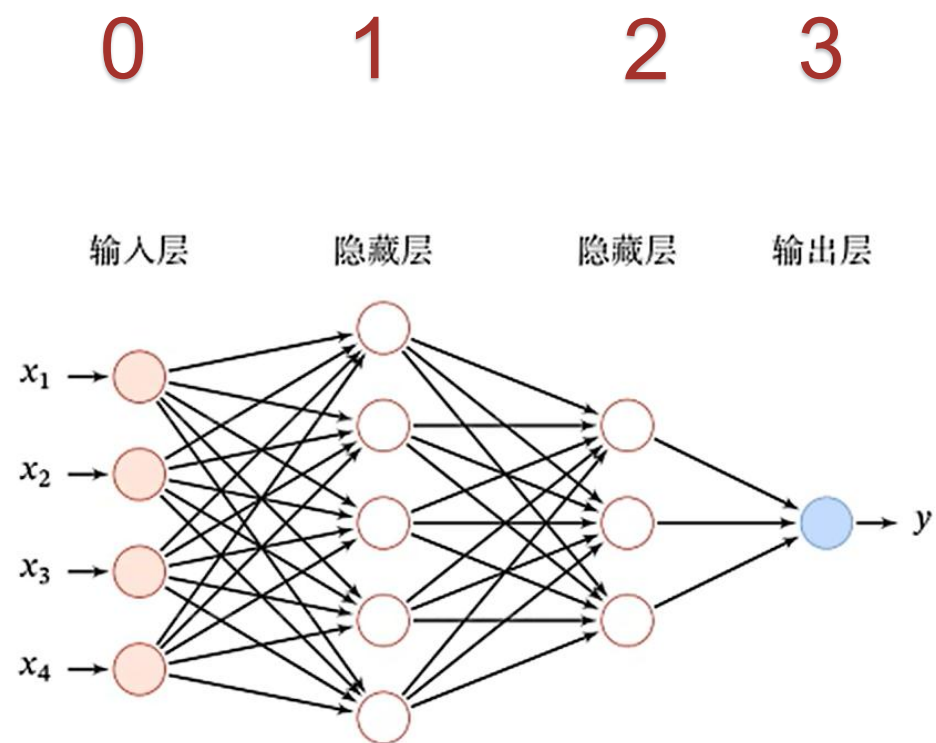
前馈神经网络

前馈神经网络的结构



前馈神经网络（Feedforward Neural Network, FNN）是最早发明的简单人工神经网络，也称为多层感知器（Multi-Layer Perceptron, MLP）。在前馈神经网络中，各神经元分别属于不同的层，每一层的神经元可以接收前一层神经元的信号，并产生信号输出到下一层，如左图所示。最左侧的一层称为输入层，最右侧一层称为输出层，其他中间层称为隐藏层。

前馈神经网络

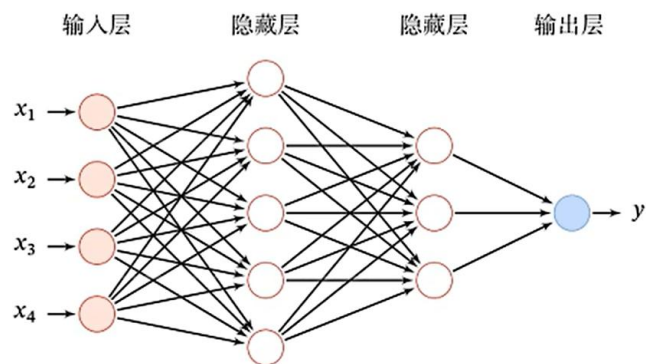


超参数 { $f_l(\cdot)$: 第 l 层的激活函数; $\hookleftarrow$
 L : 神经网络的层数; $\hookleftarrow$
 M_l : 第 l 层神经元的个数; $\hookleftarrow$

参数 { $W^l \in \mathbb{R}^{M_l \times M_{l-1}}$: 第 $l-1$ 层到 l 层的权重矩阵;
 $b^l \in \mathbb{R}^{M_l}$: 第 $l-1$ 层到第 l 层的偏置; $\hookleftarrow$

活性值 { $z^l \in \mathbb{R}^{M_l}$: 第 l 层的输入; $\hookleftarrow$
 $a^l \in \mathbb{R}^{M_l}$: 第 l 层的输出。 $\hookleftarrow$

前馈神经网络



前向传播

前馈神经网络通过不断迭代式 (5-6) 和 (5-7) 进行信息传播，这一过程被称为前向传播，这样，前馈神经网络可以通过逐层的信息传递得到最后的输出 a^L 。

记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$W^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$b^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$z^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入 (净活性值)
$a^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出 (活性值)

$$z^l = W^l a^{l-1} + b^l \quad (5-6)$$

$$a^l = f(z^l) \quad (5-7)$$



$$a^{(l)} = f_l(W^{(l)} a^{(l-1)} + b^{(l)}).$$

$$x = a^{(0)} \rightarrow z^{(1)} \rightarrow a^{(1)} \rightarrow z^{(2)} \rightarrow \dots \rightarrow a^{(L-1)} \rightarrow z^{(L)} \rightarrow a^{(L)} = \phi(x; W, b),$$

前馈神经网络

前馈神经网络具有很强的拟合能力，常见的连续非线性函数都可以用前馈神经网络来近似。

定理 4.1 – 通用近似定理 (Universal Approximation Theorem) [Cybenko, 1989; Hornik et al., 1989]: 令 $\phi(\cdot)$ 是一个非常数、有界、单调递增的连续函数, $\mathcal{J}_D$ 是一个 D 维的单位超立方体 $[0, 1]^D$, $C(\mathcal{J}_D)$ 是定义在 $\mathcal{J}_D$ 上的连续函数集合. 对于任意给定的一个函数 $f \in C(\mathcal{J}_D)$, 存在一个整数 M , 和一组实数 $v_m, b_m \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_m \in \mathbb{R}^D, m = 1, \dots, M$, 以至于我们可以定义函数

$$F(\mathbf{x}) = \sum_{m=1}^M v_m \phi(\mathbf{w}_m^\top \mathbf{x} + b_m), \quad (4.37)$$

作为函数 f 的近似实现, 即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{J}_D, \quad (4.38)$$

其中 $\epsilon > 0$ 是一个很小的正数.

根据通用近似定理，神经网络在某种程度上可以作为一个“**万能**”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。

前馈神经网络

多层前馈神经网络可以看作一个非线性复合函数 $\phi : \mathbb{R}^D \rightarrow \mathbb{R}^{D'}$, 将输入 $\mathbf{x} \in \mathbb{R}^D$ 映射到输出 $\phi(\mathbf{x}) \in \mathbb{R}^{D'}$. 因此, 多层前馈神经网络也可以看成是一种特征转换方法, 其输出 $\phi(\mathbf{x})$ 作为分类器的输入进行分类.

给定一个训练样本 $(\mathbf{x}, y)$, 先利用多层前馈神经网络将 $\mathbf{x}$ 映射到 $\phi(\mathbf{x})$, 然后再将 $\phi(\mathbf{x})$ 输入到分类器 $g(\cdot)$, 即

$$\hat{y} = g(\phi(\mathbf{x}); \theta),$$

其中 $g(\cdot)$ 为线性或非线性的分类器, θ 为分类器 $g(\cdot)$ 的参数, $\hat{y}$ 为分类器的输出.

特别地, 如果分类器 $g(\cdot)$ 为 Logistic 回归分类器或 Softmax 回归分类器, 那么 $g(\cdot)$ 也可以看成是网络的最后一层, 即神经网络直接输出不同类别的条件概率 $p(y|\mathbf{x})$.



二分类

$$p(y = 1|\mathbf{x}) = a^{(L)},$$

其中 $a^{(L)} \in \mathbb{R}$ 为第 L 层神经元的活性值.



多分类

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)}),$$

其中 $\mathbf{z}^{(L)} \in \mathbb{R}^C$ 为第 L 层神经元的净输入; $\hat{\mathbf{y}} \in \mathbb{R}^C$ 为第 L 层神经元的活性值, 每一维分别表示不同类别标签的预测条件概率.

损失函数

损失函数是用来评价模型的预测值和真实值不一样程度的函数，是一个非负实值函数，我们使用 $L(Y, f(X))$ 来表示。通常损失函数越好，模型的性能越好，而不同的模型适用的损失函数一般也不一样。

平方损失函数的标准形式如下：

$$L(Y, f(X)) = (Y - f(X))^2$$

当样本个数为 n 时，损失函数变为：

$$L(Y, f(X)) = \sum_{i=1}^n (Y - f(X))^2$$

而在实际应用中，通常会使用均方差（MSE, Mean Square Error）作为衡量指标，公式如下：

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y - f(X))^2$$

损失函数：交叉熵损失函数

对于二分类问题，交叉熵损失函数（Cross Entropy Loss）的形式如下：

$$L(Y, f(X)) = -\frac{1}{n} \sum_{i=1}^n [y_i \cdot \ln(p_i) + (1 - y_i) \cdot \ln(1 - p_i)]$$

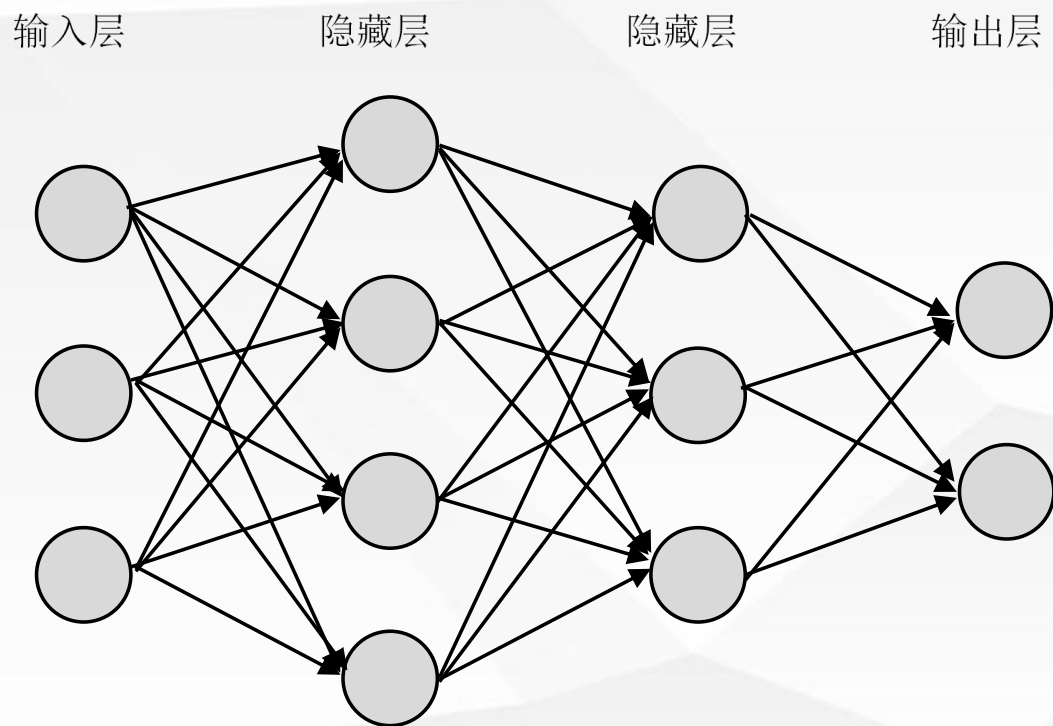
对于多分类问题，实际就是对二分类问题的扩展，其形式如下：

$$L(Y, f(X)) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{ic} \cdot \ln(p_{ic})$$

交叉熵损失函数经常用于分类问题中，特别是在神经网络做分类问题时。

前馈神经网络

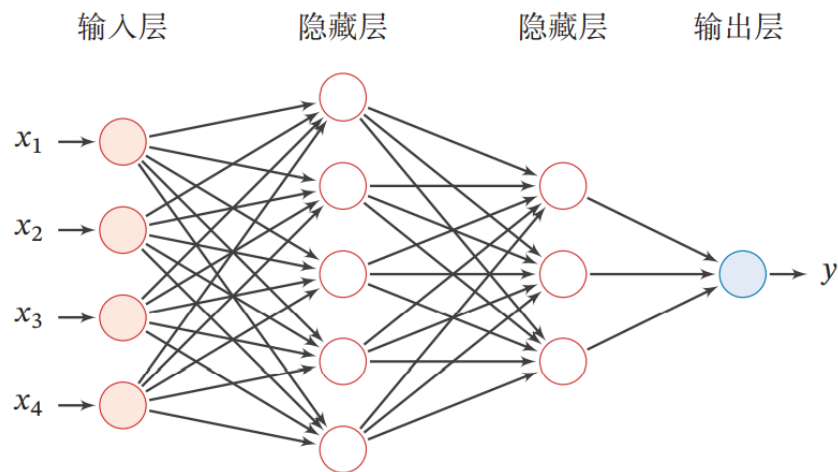
反向传播算法



在左图所示的全连接网络中，共有30个权重，神经元的每个输入都有一个权重，而第二层神经元的权重不是分配给原始输入的，而是分配给来自第一层输出的，因此，第一层权重对误差的影响是通过下一层每个神经元的权重产生的。给定一个样本 (x, y) ，将其输入到神经网络模型中，得到网络输出为 $\hat{y}$ ，则损失函数为 $L(y, \hat{y})$ ，要进行参数学习就需要将输出层得到的误差反向传播到神经网络的每一个权重，根据误差的总体变化来更新权重，这一过程就称为反向传播（Back Propagation, BP）。

。

反向传播算法



链式法则

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

► 向量关于向量的偏导数

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_N}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_M} & \cdots & \frac{\partial y_N}{\partial x_M} \end{bmatrix} \in \mathbb{R}^{M \times N}$$

若 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 为矩阵, $\mathbf{y} = g(\mathbf{X}) \in \mathbb{R}^K$, $z = f(\mathbf{y}) \in \mathbb{R}$, 则

$$\frac{\partial z}{\partial x_{ij}} = \frac{\partial \mathbf{y}}{\partial x_{ij}} \frac{\partial z}{\partial \mathbf{y}} \in \mathbb{R}.$$

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|\mathbf{W}\|_F^2,$$

$$\begin{aligned} \mathbf{W}^{(l)} &\leftarrow \mathbf{W}^{(l)} - \alpha \frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{W}^{(l)}} \\ &= \mathbf{W}^{(l)} - \alpha \left(\frac{1}{N} \sum_{n=1}^N \left(\frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{W}^{(l)}} \right) + \lambda \mathbf{W}^{(l)} \right), \end{aligned}$$

$$\begin{aligned} \mathbf{b}^{(l)} &\leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{b}^{(l)}} \\ &= \mathbf{b}^{(l)} - \alpha \left(\frac{1}{N} \sum_{n=1}^N \left(\frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{b}^{(l)}} \right) \right), \end{aligned}$$

若 $\mathbf{x} \in \mathbb{R}^M$, $\mathbf{y} = g(\mathbf{x}) \in \mathbb{R}^K$, $z = f(\mathbf{y}) \in \mathbb{R}^N$, 则

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \in \mathbb{R}^{M \times N}.$$

反向传播算法

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}},$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}.$$

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{M_l} \in \mathbb{R}^{M_l \times M_l},$$

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{M_l}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_i^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= [0, \dots, a_j^{(l-1)}, \dots, 0] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{1 \times M_l}, \end{aligned}$$

误差项

$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\begin{aligned} &= \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (\mathbf{W}^{(l+1)})^\top \cdot \delta^{(l+1)} \\ &= f'_l(\mathbf{z}^{(l)}) \odot \left((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)} \right) \in \mathbb{R}^{M_l}, \quad \text{反向传播} \end{aligned}$$

$$\mathbf{z}^{(l+1)} = \mathbf{W}^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

计算梯度

$$\begin{aligned}\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} &= \mathbb{I}_i(a_j^{(l-1)})\delta^{(l)} \\ &= [0, \dots, a_j^{(l-1)}, \dots, 0][\delta_1^{(l)}, \dots, \delta_i^{(l)}, \dots, \delta_{M_l}^{(l)}]^\top \\ &= \delta_i^{(l)} a_j^{(l-1)},\end{aligned}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)}(\mathbf{a}^{(l-1)})^\top$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

使用误差反向传播算法的前馈神经网络训练过程可以分为以下三步：

- (1) 前馈计算每一层的净输入 $\mathbf{z}^{(l)}$ 和激活值 $\mathbf{a}^{(l)}$ ，直到最后一层；
- (2) 反向传播计算每一层的误差项 $\delta^{(l)}$ ；
- (3) 计算每一层参数的偏导数，并更新参数。

算法 4.1 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α , 正则化系数 λ , 网络层数 L , 神经元数量 $M_l, 1 \leq l \leq L$.

1 随机初始化 $\mathbf{W}, \mathbf{b}$;

2 **repeat**

3 对训练集 $\mathcal{D}$ 中的样本随机重排序;

4 **for** $n = 1 \cdots N$ **do**

5 从训练集 $\mathcal{D}$ 中选取样本 $(\mathbf{x}^{(n)}, y^{(n)})$;

6 前馈计算每一层的净输入 $\mathbf{z}^{(l)}$ 和激活值 $\mathbf{a}^{(l)}$, 直到最后一层;

7 反向传播计算每一层的误差 $\delta^{(l)}$;

 // 计算每一层参数的导数

8 $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$;

9 $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$;

 // 更新参数

10 $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha (\delta^{(l)} (\mathbf{a}^{(l-1)})^\top + \lambda \mathbf{W}^{(l)})$;

11 $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \delta^{(l)}$;

12 **end**

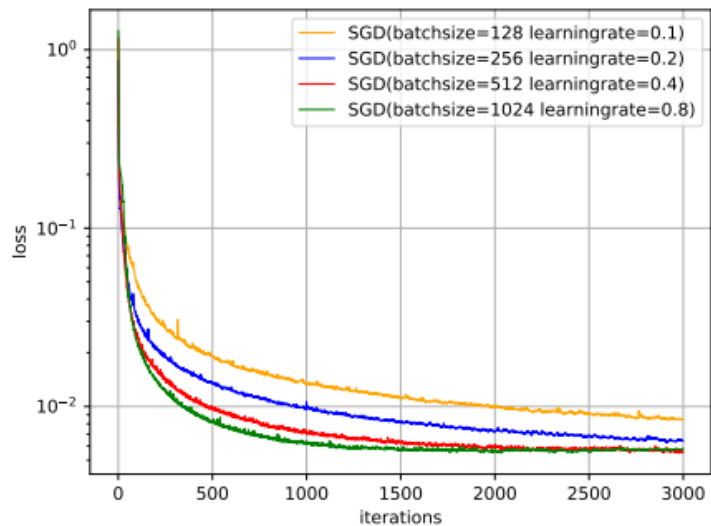
13 **until** 神经网络模型在验证集 $\mathcal{V}$ 上的错误率不再下降;

输出: $\mathbf{W}, \mathbf{b}$

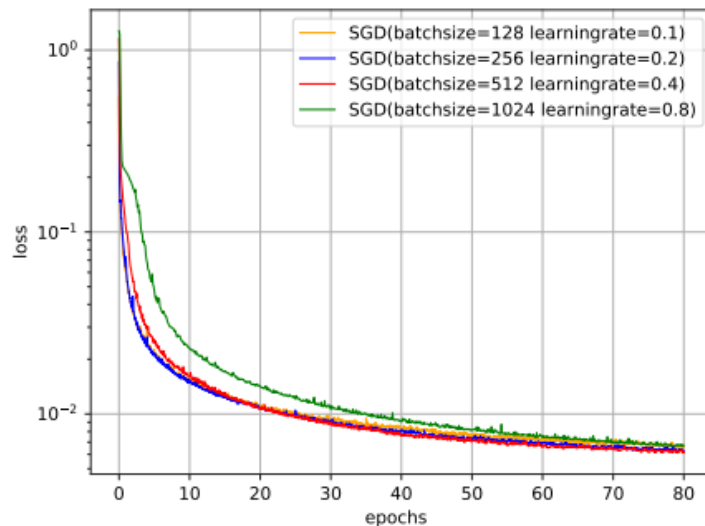
前馈神经网络

优化方法

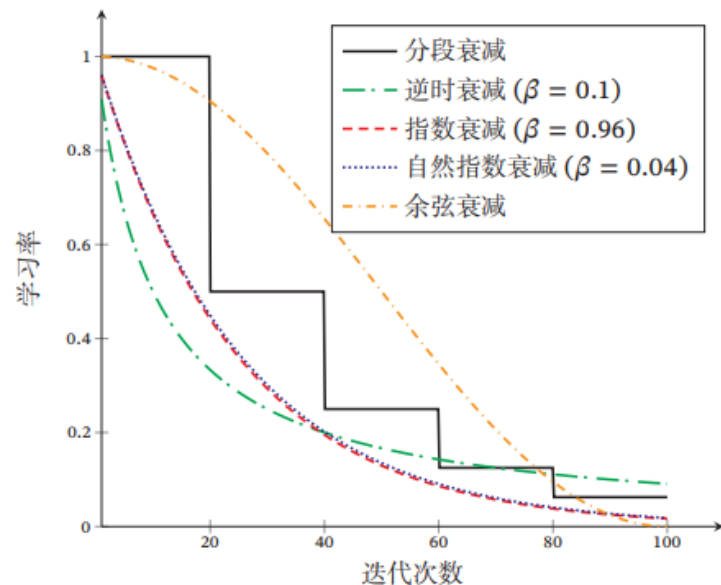
深度学习中的优化问题通常指的是寻找神经网络上的一组参数 θ ，它能显著地降低损失函数。针对此类问题，研究人员提出了多种优化算法，我们介绍其中两种比较常用的优化算法：随机梯度下降（Stochastic Gradient Descent, SGD）和自适应矩估计（Adaptive moment estimation, Adam）。



(a) 按迭代 (Iteration) 的损失变化



(b) 按回合 (Epoch) 的损失变化



前馈神经网络

优化方法：梯度下降

在机器学习算法中，在最小化损失函数时，可以通过梯度下降法来逐步迭代求解，得到最小化的损失函数和模型参数值。经典的梯度下降法在每次对模型参数进行更新时，需要遍历所有的训练数据，更新公式如下：

$$W_{k+1} = W_k - \alpha \frac{1}{n} \sum_{i=1}^n g_k^i$$

前馈神经网络

优化方法：随机梯度下降

当样本规模很大的时候，梯度下降法需要耗费巨大的计算资源和计算时间，这在实际过程中基本不可行。为了解决这个问题，随机梯度下降法应运而生。

SGD使用单个随机样本的梯度作为方向，步长作为一个可调控因子随时调整，可表示为如下形式：

$$W_{k+1} = W_k - \alpha g_k^i$$

前馈神经网络

优化方法：小批量梯度下降

SGD每次迭代只使用一个样本计算梯度，训练速度快。但是如果数据的分布极不均衡，包含大量噪声，会造成损失函数在收敛过程中产生严重震荡。因此，一个自然而然的想法就是每次使用一批数据（batch）来计算梯度，然后将梯度平均之后再使用SGD去更新参数，通过平均一批样本的梯度结果来减少震荡和其他影响，这种方法称之为小批量（mini-batch）梯度下降法，其更新公式如下：

$$W_{k+1} = W_k - \alpha \frac{1}{m} \sum_{j=1}^m g_k^{i_j}$$

优化方法：Adam优化方法

Adam是对SGD的扩展，可以代替SGD来更有效地更新网络权重。Adam 算法很容易实现，并且有很高的计算效率和较低的内存需求，很适合求解带有大规模数据或参数的问题。其具体的计算方法我们不做介绍，感兴趣的读者可以自行查阅相关资料。

类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam≈动量法 +RMSprop

正则化技术

Regularization Techniques

防止过拟合，提升模型泛化能力

过拟合与 Dropout

什么是过拟合?

模型在训练集上表现很好, 但在新数据上表现很差。

原因: 模型 “记住” 了训练数据的噪声, 而不是学到真正的规律。

解决思路: 限制模型复杂度, 让它 “不要记得太细”。

Dropout (2012, Hinton)

训练时: 每次前向传播随机 “丢弃” 一部分神经元 (设为0)

测试时: 使用所有神经元, 但输出乘以保留概率 $(1-p)$

$h = \text{Dropout}(h, p=0.5)$

直觉: “不让网络依赖任何单一神经元, 迫使它学习冗余特征”

其他正则化方法

L1/L2 正则化

在损失函数中加入权重的范数惩罚项, L2也叫 Weight Decay, 限制权重大小。

Early Stopping

监控验证集损失, 当它不再下降时停止训练, 防止模型 “学过头”。

数据增强

通过翻转、旋转、裁剪、加噪等扩充训练数据, 提升泛化能力。

模型集成

训练多个模型, 将结果平均/投票, Dropout可视为难式集成。

Batch Normalization (批量归一化)

核心理念

2015年 Ioffe & Szegedy 提出，解决深层网络训练困难的问题。

对每一层的输入进行归一化：

$$\hat{x} = (x - \mu_B) / \sqrt{\sigma^2_B + \epsilon}$$
$$y = \gamma \times \hat{x} + \beta$$

其中 μ_B 和 σ^2_B 是当前 batch 的均值和方差， γ 和 β 是可学习的缩放和平移参数。

为什么有效？

- ✓ 缓解内部协变量偏移 (ICS)
- ✓ 允许使用更大的学习率
- ✓ 加速收敛，减少训练时间
- ✓ 具有轻微的正则化效果
- ✓ 减少对初始化的敏感度

PyTorch:

```
nn.BatchNorm1d(num_features)
```

Batch Norm vs Layer Norm 对比

维度

归一化方向
依赖 batch
主要用途

Batch Normalization

沿 batch 维度 (同一特征的不同样本)
是 (batch太小时不稳定)
CNN、图像任务

Layer Normalization

沿特征维度 (同一样本的不同特征)
否 (与 batch 大小无关)
Transformer、NLP、RNN

权重初始化策略

为什么初始化很重要？

全零初始化：所有神经元学到相同特征，失去“对称性破坏”

过大初始化：梯度爆炸，训练不稳定

过小初始化：梯度消失，学习缓慢

原则：保证每层输出的方差与输入一致，避免信号在传播过程中放大或缩小。

实践指南

- ✓ 用 ReLU / GELU → 选 He (Kaiming) 初始化
- ✓ 用 Sigmoid / Tanh → 选 Xavier (Glorot) 初始化
- ✓ 用了 Batch Norm / Layer Norm → 初始化的影响会减小，但仍推荐正确初始化
- ✓ PyTorch 的 `nn.Linear` 默认使用 Kaiming 初始化，大多数情况无需手动设置

两大经典初始化方法

Xavier 初始化 (Glorot, 2010)

适用：Sigmoid / Tanh 激活函数

$$W \sim U[-\sqrt{6/(n_{in}+n_{out})}, \sqrt{6/(n_{in}+n_{out})}]$$

He 初始化 (Kaiming He, 2015)

适用：ReLU 及其变体

$$W \sim N(0, 2/n_{in})$$

PyTorch:

```
nn.init.xavier_uniform_(layer.weight)
nn.init.kaiming_normal_(layer.weight)
```


课程目录

Course catalogue

1/ 深度学习与神经网络

2/ 人工神经网络

3/ 前馈神经网络

4/ 深度学习框架

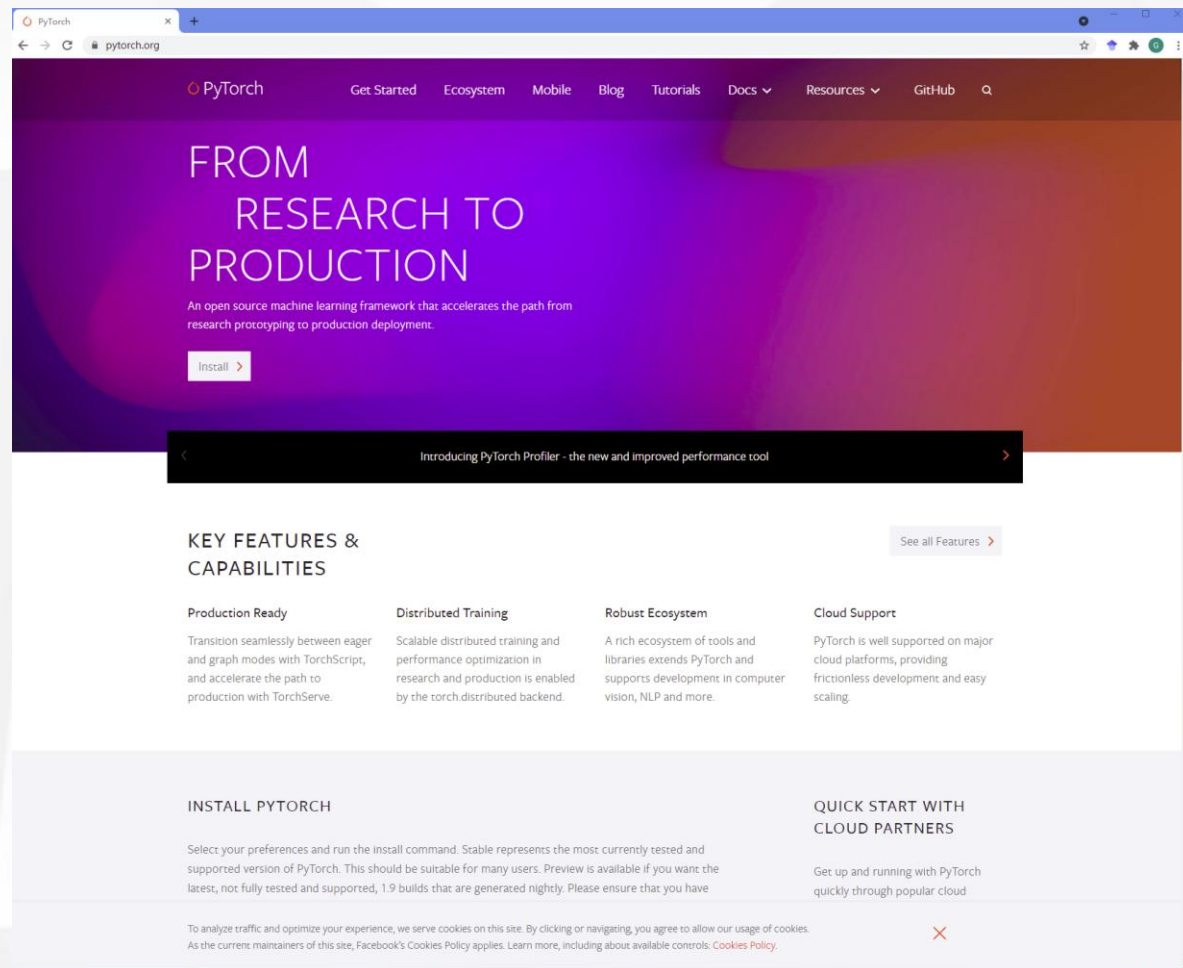
深度学习框架

深度学习框架

当我们需要实现神经网络时，使用深度学习框架进行开发可以为我们省去大量而繁琐的外围工作，方便实现神经网络的构建和训练。使用深度学习框架完成模型构建主要有两个优势：

- 屏蔽底层实现，用户只需关注模型的逻辑结构，可以节省大量编写底层代码的精力；
- 具备灵活的移植性，方便将代码部署到CPU、GPU或移动端上。

深度学习框架



Pytorch

PyTorch是基于Torch并由Facebook强力支持的python端的开源深度学习库。它于2017年1月发布，是近年来最引人关注的深度学习框架之一。2018年Caffe2正式并入PyTorch后，PyTorch的发展势头更呈不可阻挡之势。

PyTorch 实战：构建一个 MLP

完整代码示例

```
import torch
import torch.nn as nn

# 1. 定义模型
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 256),
            nn.BatchNorm1d(256),
            nn.GELU(),
            nn.Dropout(0.3),
            nn.Linear(256, 10))
    def forward(self, x):
        return self.net(x)

# 2. 训练配置
model = MLP()
optimizer = torch.optim.AdamW(
    model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
```

代码解读

nn.Sequential

按顺序堆叠层，就像“流水线”一样

nn.Linear(784, 256)

全连接层，784输入 → 256输出

nn.BatchNorm1d(256)

批量归一化，加速训练

nn.GELU()

现代激活函数，Transformer标配

nn.Dropout(0.3)

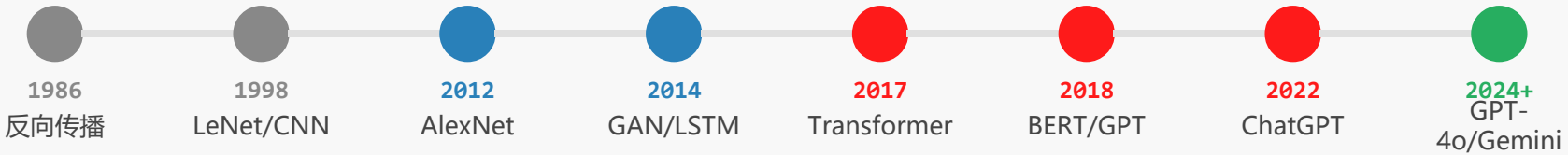
随机丢弃30%神经元，防过拟合

AdamW

Adam + Weight Decay，当前最常用的优化器

从神经网络到大模型时代

发展路线



Transformer 架构

2017年 Google 提出 "Attention Is All You Need"

- 核心创新:
- ✓ 自注意力机制 (Self-Attention)
允许模型关注输入序列的任意位置
 - ✓ 多头注意力 (Multi-Head Attention)
 - ✓ 位置编码 (Positional Encoding)
 - ✓ 并行计算, 取代 RNN 的顺序处理

大语言模型 (LLM)

基于 Transformer 的超大规模预训练模型:

- GPT 系列 (OpenAI)**
自回归生成模型, 预测下一个 token
- BERT (Google)**
双向编码器, 擅长理解任务
- LLaMA (Meta) / Qwen (阿里) / DeepSeek**
开源大模型, 推动学术和工业界发展

这些模型的基础 = 本章所学的神经网络!

结束

谢谢